PROJECT AND TEAM INFORMATION

Project Title

Load Balancer: Intelligent load balancing and resource allocation

Student / Team Information

Team Name: *Team #*	*NEXORA*
Team member 1 (Team Lead) *(Raj, Amrit: 2510010756: amrit2202raj@gmail.com):*	*Raj, Amrit – 2510010756* *amrit2202raj@gmail.com* 
Team member 2 *(Gupta, Pawani: 2510380107: pawanigupta030@gmail.com):*	*Gupta, Pawani – 2510380107* *pawanigupta030.com* 
Team member 3 *(Shandilya, Anvi: 2510380201: 8fanvibis.shandil@gmail.com):*	*Shandilya, Anvi – 2510380201* *8fanvibis.shandil@gmail.com* 
Team member 4 *(Kumar, Rushil: 2510350004: kumarrushil21@gmail,com):*	*Kumar, Rushil – 2510350004* *kumarrushil211@gmail.com* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

Servers are used by huge amount of population to do different tasks. Sometimes, one server gets much off tasks and becomes slow. At the same time, other servers may have less tasks and are faster as compared to the slower one.Load balancer solves this problem. It will send tasks to different servers according to task type so that the work is shared between them. Before sending a task, the system will check the server's CPU, RAM, and current work. If a server is too busy, the task will be sent to another server. If no server is free, the task will wait in queue until a server is available. We chose this topic because it is a simple way to understand how cloud servers manage many tasks. It will also help us use the DSA and OOP with C++ concepts in this project.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Servers use load balancing methods to share tasks between different servers. A load balancer receives requests from users and sends them to the required servers. The system checks the work load on each server before sending a task.If a server is busy or does not have enough resources or space, the task can be sent to another server or will be in queue in waiting. These methods help servers handle tasks and avoid work overflow on one server. Our system will use these ideas in a simple way. It will check the server load and available resources before giving a task and will keep tasks waiting when any server is not server is available.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

The main goal of our project is to make a load balancer system that can share traffic between different servers. The system will check the work, CPU, and RAM of each server before sending traffic. It will send tasks to the server that can handle properly according to their requirments and keep a task waiting if no server has enough resources. The first step is to decide the number of servers, their resources, task types, and basic system design. Next, we will create the Task and Server classes and make the task queue. After this, we will build the load balancing system and add CPU and RAM checking for task allocation. We will then add the waiting queue and release the resources when a task is completed. Finally, we will connect all parts and test the complete system using different numbers of servers and tasks.

Project Approach (3 pts)

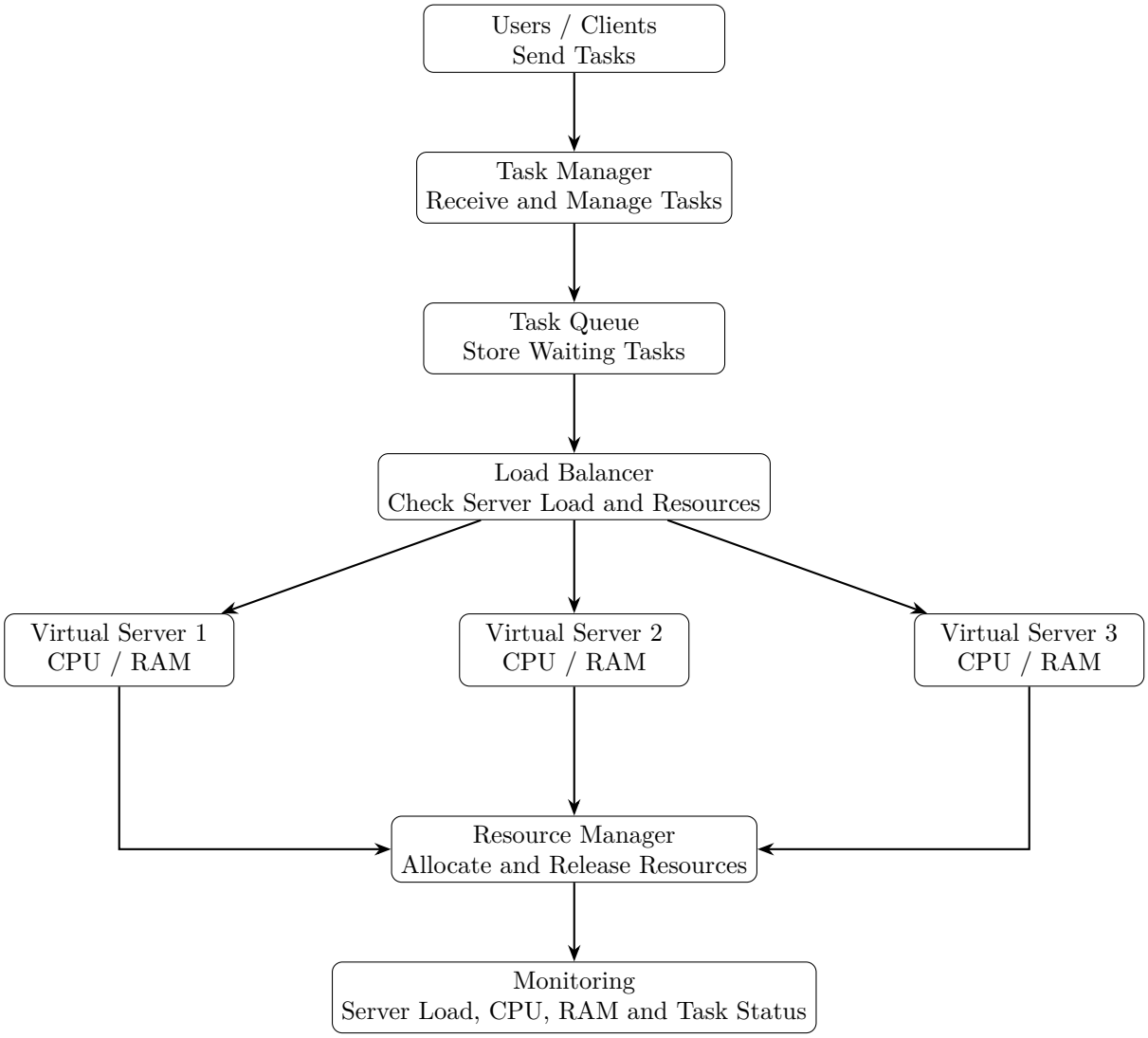
(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

–> We will make a simple system to manage tasks and cloud servers.
–> We will create different servers with fixed CPU and RAM.
–> We will create tasks with different CPU and RAM needs.
–> Tasks will be stored in a queue until they are assigned.
–> The system will check the work and available resources of each server.
–> The load balancer will choose a suitable server.
–> The task will be sent to that server.
–> If no server has enough resources, the task will stay in the queue.
–> When a task is finished, its resources will be given back to the server.
–> We will use C++ to build the project.
–> We will use OOP concepts to make classes like Server, Task, and Load Balancer.
–> We will use DSA concepts like queues, priority queues, searching, and sorting.
–> We will keep the system simple and easy to test.
–> Finally, we will test it with different numbers of servers and tasks.
–> We will check server load, available resources, waiting tasks, and completed tasks.

System Architecture (High Level Diagram)(2 pts)

*(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool
of your choice).*

User –> Task Manager –> Task –> Queue –> Load Balancer –> Virtual Servers –> Resource manager –> Monitoring



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

–> Make a cloud system that can manage tasks and server resources.
–> Users can send tasks to the system.
–> The system will send tasks to different cloud servers.
–> The load balancer will check which server is free.
–> It will send the task to a suitable server.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

–> Servers have fixed CPU and RAM.
–> Different tasks have different CPU and RAM needs.
–> There is more than one server to share the tasks.
–> Server information is available to the load balancer.
–> A task needs enough CPU and RAM to run.
–> If no server has enough resources, the task waits in the queue.
–> Resources are given back when a task is finished.
–> The system will be simple.
–> The system will be tested with a limited number of servers and tasks.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

1.GeeksforGeeks - Cloud Computing: https://www.geeksforgeeks.org/cloud -computing
2.AWS - Load Balancing: https://aws.amazon.com/what-is/load -balancing
3.Microsoft Azure - Load Balancer: https://azure.microsoft.com/en-us/products /load-balancer
4.Google Cloud - Load Balancing: https://cloud.google.com/load-balancing